

Summary

This video explain key technical concepts from **Database Management Systems (DBMS)** with a focused lens on **Functional Dependencies (FDs)** and **Normalization**, aligned with the **GATE 2026** curriculum.

The discussion progresses from foundational principles such as **key identification** and **FD properties** to advanced analytical constructs including **membership (logical implication) of FDs**, **equivalence of FD sets**, and the systematic derivation of **Minimal (Canonical) Covers**. It further examines **table decomposition**, distinguishing **lossless** from **lossy** decompositions, and explains the critical notion of **dependency preservation**.

Critical Takeaways

- **Key Identification:** A Super Key is defined by its closure containing all attributes of a relation; a Candidate Key is the minimal such set.
- **Minimal Cover:** A three-step optimization process—RHS splitting, LHS simplification, and redundancy elimination—used to simplify FD sets without changing their implication power.
- **Lossless Decomposition:** A decomposition is lossless if the intersection of decomposed relations is a super key for at least one of them.
- **Dependency Preservation:** A decomposition preserves dependencies if all original FDs can be enforced using only the decomposed relations.

1. Fundamental Review: Keys and Attributes

Before advancing to complex FD operations, the following foundational concepts must be clearly established.

1.1 Key Definitions

Super Key

An attribute or set of attributes is a Super Key if its closure (X^+) contains all attributes of the relation.

Candidate Key

A minimal Super Key. No proper subset of a Candidate Key can uniquely identify all attributes.

Prime Attribute

An attribute that is part of at least one Candidate Key.

Non-Prime Attribute

An attribute that does not belong to any Candidate Key.

1.2 Functional Dependency Properties

The following seven inference rules (Armstrong's Axioms and derived rules) govern reasoning with FDs:

1. **Reflexivity**
If $Y \subseteq X$, then $X \rightarrow Y$.
2. **Augmentation**
If $X \rightarrow Y$, then $XZ \rightarrow YZ$.
3. **Transitivity**
If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$.
4. **Union**
If $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$.
5. **Decomposition**
If $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$.
6. **Pseudo-Transitivity**
If $X \rightarrow Y$ and $WY \rightarrow Z$, then $WX \rightarrow Z$.
7. **Composition**
If $X \rightarrow Y$ and $A \rightarrow B$, then $XA \rightarrow YB$.

2. Logical Implication and Membership Sets

A **Membership Set** (or logically implied FD) exists when a functional dependency can be derived from a given FD set using inference rules.

Determination Method

To verify whether $X \rightarrow Y$ is logically implied by a set of FDs:

- Compute the closure X^+ using the given FD set.
- If $Y \subseteq X^+$, then $X \rightarrow Y$ is a member of the implied FD set.

Example Insight

To check whether $C \rightarrow G$ is implied:

- Compute C^+ .
- If attribute G appears in C^+ , then $C \rightarrow G$ is logically implied

3. Equivalence of FD Sets

When comparing two FD sets, F and G, four relationships are possible:

1. **F covers G**
Every FD in G can be derived from F, i.e., $G \subseteq F^+$.
2. **G covers F**
Every FD in F can be derived from G, i.e., $F \subseteq G^+$.
3. **Equivalence ($F \equiv G$)**
F covers G and G covers F.
4. **Non-Equivalence**
Neither set fully covers the other.

Verification Process (Does F cover G?)

- For each FD $X \rightarrow Y$ in G:
 - Compute X^+ using only F.
 - If $Y \subseteq X^+$ for all such dependencies, then F covers G.
- If even one FD in G fails this test, F does not cover G.

4. Minimal Cover (Canonical Cover)

A **Minimal Cover** is a simplified and non-redundant representation of an FD set that preserves all logical implications.

Three-Step Procedure for Finding a Minimal Cover

Step 1: Split RHS

Ensure that every FD has exactly one attribute on the right-hand side using decomposition.

Example:

$E \rightarrow AD$ becomes

$E \rightarrow A$ and $E \rightarrow D$

Step 2: Simplify LHS

If an FD has multiple attributes on the LHS, check for redundancy.

For $AC \rightarrow D$:

- Compute A^+ .
- If $C \in A^+$, then C is redundant and the FD simplifies to $A \rightarrow D$.

Step 3: Remove Redundant FDs

Apply the “Hide and Seek” method:

- Temporarily remove one FD.
- Try to derive its RHS from the remaining FDs.
- If derivable, the FD is redundant and permanently removed.

5. Table Decomposition

Decomposition divides a large relation into smaller relations to reduce redundancy and anomalies while preserving correctness.

5.1 Lossless vs. Lossy Decomposition

Lossless Join Decomposition

A decomposition is lossless if joining the decomposed relations yields exactly the original relation.

Condition

Let a relation R be decomposed into R_1 and R_2 .

Let $X = R_1 \cap R_2$.

If:

- X is a Super Key of R_1 , or
- X is a Super Key of R_2 ,
then the decomposition is lossless.

Verification Method

- Find the common attributes $R_1 \cap R_2$.
- Compute the closure of the common attributes.
- If the closure contains all attributes of R_1 or R_2 , the decomposition is lossless.

Lossy Decomposition

If the common attributes are not a super key for either relation, joining R_1 and R_2 produces spurious tuples, resulting in incorrect or extra data.

5.2 Dependency Preservation

A decomposition is **Dependency Preserving** if all original functional dependencies can be enforced using only the decomposed relations, without performing joins.

Significance

- Ensures that all business rules and integrity constraints remain enforceable.
- Improves efficiency and maintainability of the database schema.

Verification

- For each original FD $X \rightarrow Y$:
 - Check whether it exists entirely within a single decomposed relation.
 - If not, attempt to derive it using closures across decomposed relations, possibly applying transitivity or pseudo-transitivity.
- If all original FDs can be derived, the decomposition is dependency preserving.

